Homework — 1.4.2 Data Structures — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Total: Part A 14 + Part B 6 = 20 marks.

Part A

1. (a) [2] Completed stack table:

Operation	Stack after the operation (top first)	Value popped
`push(8)`	8	—
`push(3)`	3, 8	—
`push(6)`	6, 3, 8	—
`pop()`	3, 8	**6**
`pop()`	8	**3**
`push(1)`	1, 8	—

Mark: all six rows fully correct = 2; four or five rows correct = 1. A row is correct only if both the contents and the popped value are right.

(b) [2] Completed queue table:

Operation	Queue after the operation (front first)	Value dequeued
`enqueue(4)`	4	—
`enqueue(9)`	4, 9	—
`dequeue()`	9	**4**
`enqueue(2)`	9, 2	—
`dequeue()`	2	**9**

Mark: all five rows fully correct = 2; three or four rows correct = 1. Items must join at the rear and leave from the front (FIFO).

2. [3] Completed After table:

Address	Data	Next
100	5	200
200	12	**400**
300	20	null
400	15	**300**

Mark: node 400 holds data 15 with next = 300 (i.e. it takes node 200's old pointer) (1); node 200's next changed to 400 (1); nodes 100 and 300 completely unchanged — no data moves in memory, only pointers change (1).

3. (a) [2] Completed BST table:

Value	Parent node	Left or right child of its parent?
40	— (root)	—
60	40	right (60 > 40)
25	40	left (25 < 40)
35	25	right (35 > 25, and 35 < 40)
10	25	left (10 < 25)
50	60	left (50 < 60, and 50 > 40)

Mark: all five completed rows correct = 2; three or four correct = 1.

(b) [1] In-order (Left, Root, Right): **10, 25, 35, 40, 50, 60** — the ascending/sorted order (1). *(AO2)*

4. [2] Completed true/false table:

Statement	True / False	Correction if false
A queue is a LIFO structure.	**False**	A queue is **FIFO** — first in, first out (a stack is LIFO).
An in-order traversal of a binary search tree visits the values in ascending order.	**True**	—
Inserting a value into a linked list means all the following nodes must be moved in memory.	**False**	No nodes move — only the **pointers** are changed (the new node takes the previous node's pointer, and that pointer is redirected to the new node).
Pushing an item onto a stack that is already full causes a stack overflow.	**True**	—

Mark: 2 correct rows per mark (4 rows → 2 marks). A false row scores only if a valid correction is given.

5. [2] 23 MOD 10 = **3**; 57 MOD 10 = **7**; 33 MOD 10 = **3** (1). **Collision:** 33 hashes to index 3, which is already occupied by 23 — plus a valid resolution method named (1), e.g. **linear probing** (place at the next free slot) or **chaining** (store a linked list of keys at that index). *(AO2)*

Part B

6. [2] Interrupts can occur while another interrupt is already being serviced (nested), so the CPU must be able to return to each task in the correct order (1). A stack is **LIFO**, so the registers (e.g. PC and ACC) of the interrupted task are **pushed** on, and when the higher-priority ISR finishes they are **popped** off in reverse order — the most recently interrupted task resumes first — guaranteeing every task resumes correctly even with several interrupts nested (1). *(AO1)*

7. [1] The control bus carries **control signals** (e.g. read/write signals, clock pulses, interrupt requests) between the CPU and other components to coordinate/synchronise their operation. *(AO1)*

8. [3] A foreign key is an attribute in one table that is the **primary key of another table**, used to **link** the two tables together (1). Referential integrity ensures that every foreign-key value **matches a valid existing primary key** in the related table (or is null) (1), so there are **no "orphaned" records** that reference a row which does not exist (1). *(AO1)*